Journal of Bioinformatics and Computational Biology
© Imperial College Press

Kernel-Based Self-Organized Maps trained with Supervised Bias for Gene Expression Data Analysis

STERGIOS PAPADIMITRIOU*

Department of Information Management, Technological Education Institute of Kavala, 65404 Kavala, Greece, email: sterg@teikav.edu.gr

SPIRIDON D. LIKOTHANASSIS

Pattern Recognition Laboratory, Department of Computer Engineering and Informatics, School of Engineering, University of Patras, Rion, Patras 26500, Greece, email: likothan@ceid.upatras.gr
and
Computer Technology Institute, 3 Kolokotroni street, 262 21 Patras, Greece, email: likothan@cti.gr

Received (Day Month Year)
Revised (Day Month Year)
Accepted (Day Month Year)

Self-Organized Maps (SOMs) are a popular approach for analyzing genome-wide expression data. However, most SOM based approaches ignore prior knowledge about functional gene categories. Also, Self Organized Map (SOM) based approaches usually develop topographic maps with disjoint and uniform activation regions that correspond to a hard clustering of the patterns at their nodes. We present a novel Self-Organizing map, the Kernel Supervised Dynamic Grid Self-Organized Map (KSDG-SOM). This model adapts its parameters in a kernel space. Gaussian kernels are used and their mean and variance components are adapted in order to optimize the fitness to the input density. The KSDG-SOM also grows dynamically up to a size defined with statistical criteria. It is capable of incorporating a priori information for the known functional characteristics of genes. This information forms a supervised bias at the cluster formation and the model owns the potentiality of revising incorrect functional labels. The new method overcomes the main drawbacks of most of the existing clustering methods that lack a mechanism for dynamical extension on the basis of a balance between unsupervised and supervised drives.

Keywords: Gene Expression Analysis, Clustering, Self-Organized Maps, Entropy, Supervised Bias, Genome Data Mining

1. Introduction

Data Mining techniques are very popular in Bioinformatics [1,2]. Clustering is one of the most widely used data analysis technique for genome-wide expression data.

*Corresponding author

It aims to provide insight into the structure of the data and aids at the discovery of functional classes. Gene expression analysis, utilizes clustering techniques extensively. These techniques accomplish the grouping of genes with similar expression patterns into clusters [3,4,6,7,8]. Such approaches unravel relations between genes and help to deduce their biological role, since genes of similar function tend to display similar expression patterns.

Most of the so far developed algorithms perform the clustering of the expression patterns [a] in an unsupervised manner [4,9,10], although in many cases already exists valuable domain knowledge. For example, collections of genes known to encode proteins of similar biological function, e.g. genes that code for ribosomal proteins, constitute useful a priori knowledge [11]. This means that existing information is not fully explored in order to deduce the correct expression characteristics of genes that make them part of functional groups. Additionally the frequent case that genes of similar function become allocated to different clusters and are therefore known to be erroneously grouped cannot be handled by a pure unsupervised approach. Despite of the need to integrate a priori knowledge, most of the widely used clustering methods, like hierarchical clustering [4,5], K-means clustering, Bayesian clustering [12,13], supervised harvesting of expression tress [14] and the Self-Organizing Map (SOM) [10] usually ignore any existing class information. In addition many kernel-based developments of the SOM approach, do not have an explicit provision for incorporating supervised labeling [15,16].

The standard SOM algorithm has a number of properties, which render it to a candidate of particular interest as a basis framework for building more flexible and advanced algorithms for massive genome-wide data analysis. SOMs can be implemented easily, are fast, robust and scale well to large data sets. They allow one to impose partial structure on the clusters and facilitate visualization and interpretation. In the case hierarchical information is required, it can be implemented on top of SOM, as in [17].

Recently, several dynamically extended schemes have been proposed that overcome the limitation of the fixed non-adaptable architecture of the SOM. Some examples are the Dynamic Topology Representing structures [18], the Growing Cell Structures [13,19], Self-Organized Tree Algorithms [20,9], the joint entropy maximization approach [21] and the Adaptive Resonance Theory [22,23].

The presented approach has some similarities to these dynamically extended schemes, from the point of view of its unsupervised component. However, one essential difference exists: all the forementioned schemes are purely unsupervised, lacking a design for the incorporation of problem domain knowledge. Instead, we focus on the design of such types of algorithms that aim to explore effectively existing *a priori supervised class labeling*, for *multi-class* and *multi-labeled* gene expression data. The multiple labeling, i.e. the possible assignment of more than one class label at

[a] In the context of the current paper by pattern we refer to the expression profile of a gene recorded over a set of experiments

each gene, perplexes the clustering and classification tasks. The multiple functional labeling of genes is the rule and not the exception, i.e. most genes belong to more than one functional class.

Also, in contrast to the complexity of some of these schemes, we design simple algorithms that through the restriction of growing on a rectangular grid, can be implemented easily and the training of the models is very efficient. In addition, most of the benefits of the more complex alternatives of dynamical extension are still retained.

We call the proposed model KSDG-SOM from Kernel Supervised Dynamic Grid SOM, since it is a model trained in kernel space and although it is SOM based it tightly integrates unsupervised and supervised learning components.

As a kernel the Gaussian one is used. The Gaussian kernel mapping implements more elaborate soft class separation boundaries than the hard separation onto Voronoi regions obtained by evaluating directly at the input space the inner products of the patterns and the prototype vectors. As with other kernel methods [24,25], we aim to exploit a nonlinear transformation of the input space onto a high-dimensional feature space. Intuitively, the SOM based learning constructs Voronoi regions over this high-dimensional space in which the extra dimensions enhance the possibilities of defining more elaborately the cluster boundaries.

The KSDG-SOM has been designed in order to automatically detect the *appropriate level of expansion*, so that the number of clusters is controlled by a properly defined measure of the algorithm itself, with no need for any a priori specification. This is quite important for gene expression data analysis tasks where very little (or nothing) can be claimed about an a priori estimate of the number of clusters.

Unfortunately, the functions of most genes are either unknown or partially and unreliably known (in fact the discovery of functional information is one of the main objectives of gene expression analysis). Motivated by this we have designed the KSDG-SOM in order to be simply another one clustering and classification device but instead and more important to be *a reclassification device*. It takes both input data and existing classification labels into account in order to evaluate a new set of classes. The cost function (minimized during learning) implements a trade-off between the unsupervised quantization error (arising from the input variance) and the supervised mislabeling error (quantified by an entropy-like term). The unsupervised error in traditional supervised approaches (e.g. Radial Basis Function, Support Vector Machines) is hidden behind the supervised error, i.e. the unsupervised techniques are used as hidden steps, not directly connected to the ouput labels. Hence, these approaches *cannot penalize explicitly the dispersion in the input space*. The presented approach accomplishes such a compromise between unsupervised and supervised drives that guides to a reclassification of the training set, that can expose new insights at the problem structure. In comparison, the supervised machinery of the Support Vector Machines (SVMs) has been used successfully to detect possible isolated functional gene labeling errors at the training set of microarray based expression experiments by implicitly constructing the mapping from the

gene expression profile to the functional category [11]. However, SVMs cannot implement a drastic reorganization of functional class assignments, and their performance degrades severely when the functional labelings at the training set are unreliable.

Comparing with other related approaches in the literature, that attempt to integrate both unsupervised and supervised learning, the approach of [26] preserves the topology of the space defined by the patterns themselves ("primary space" e.g. unsupervised part) and uses the class labelings ("auxiliary" data) to measure distances by similarity in the auxiliary space. The locality criterion in terms of the primary space implements the unsupervised contribution, while the computation of similarities in terms of the labeled "auxiliary" space constitutes the supervised part. However, such an approach is aimed for the cases where sufficient and reliable class information exist. To the contrary, we aim to exploit supervised information only when it exists and thus we confront the problems with both incomplete and unreliable class labelings of gene expression data.

Another related work, is the approach to reclassification with supervised clustering presented in [27]. The main differences and improvements of our approach compared with the former are:

(1) We perform the determination of the proper number of clusters for each unsupervised/supervised relative strength via the dynamical extension process of the KSDG-SOM that is controlled with rigid statistical criteria.
(2) We exploit the advantages of the kernel mapping. We prefer the use of Gaussian kernels due to their mathematical tractability.
(3) We quantify the inconsensus in supervised labelings of the patterns represented by a node with an entropy-like term that is descriptive even for the symbolic unordered class labelings of the genes.
(4) We confront the case of multiple functional class labelings of the genes.

The paper is outlined as follows: Section 2 reviews briefly the gene expression measurements with DNA microarrays. Section 3 deals with the definition of error measure minimization in kernel space. Section 4 derives on the strict basis of maximum likelihood an error criterion (i.e. a cost function) for the integration of supervised/unsupervised information. Section 5 deals with the learning algorithms that adapt both the structure and the parameters of the KSDG-SOM. The expansion phase of the KSDG-SOM learning is described in Section 6 separately since it is rather lengthy. Section 7 deals with the automatic detection of the appropriate expansion level. Section 8 discusses results obtained from an application of the KSDG-SOM to gene expression data analysis tasks. The results demonstrate the reclassification potentiality of the model and compare its performance with other competitive machine learning approaches both for clustering and classification tasks. Finally, Section 9 presents the conclusions along with some directions onto which further research can proceed for improvements. Some algorithmic details of the algorithms are presented with Appendix A.

2. DNA microarray data

The DNA microarray technology provides the ability to measure the expression levels of thousands of genes in a single experiment [3,11,28,4]. The interpretation of such massive expression data is a new challenge for bioinformatics and opens new perspectives for functional genomics. A key question within this context is if given some expression data for a gene, this gene does belong to a particular *functional class* (i.e. it encodes for a protein of interest).

The data points produced by the DNA microarray hybridization experiments represent the ratio of expression levels of a particular gene under two different experimental conditions. During the microarray construction, several thousand DNA samples are fixed to a glass slide, each at a known position in the array. Each such sample corresponds to a single gene of the organism being investigated. Subsequently, messenger RNA samples are collected from a population of cells subjected to various experimental conditions. By means of reverse transcription these samples are converted to cDNA and are labeled with one of two different fluorescent dyes. A single experiment consists of hybridizing the microarray with two differently labeled cDNA samples collected at different times. Usually, one of the samples is from the reference or background state of the cell, while the other sample represents a special condition set up by the experimenter (i.e. a drug administration). The level of expression of a particular gene is roughly proportional to the amount of cDNA that hybridizes with the DNA affixed to the slide. Laser scanning technology is used to measure the ratio of the two dyes present at the position of each DNA sequence. This ratio yields the relative levels of gene expression for any pair of measured conditions.

Therefore, the result from an experiment with n DNA samples on a single microarray, is a series of n expression-level ratios. Typically, the numerator of each ratio is the expression level of the gene in the condition of interest at the experiment, while the denominator is the expression level of the gene in the reference state of the cell. The data from a series of m such experiments are usually represented as a gene expression matrix, in which the rows represent the m-element expression vectors of each gene.

Gene expression data are characterized by *multiple functional labeling* that is however still very incomplete even for the simpler and more extensively studied organisms as the yeast. Thus, the available a priori knowledge is still very unreliable. We have developed the KSDG-SOM in order to confront effectively the gene expression data exploratory analysis. We proceed by describing the design and the algorithms of the model.

3. Kernel Space Adaptation of Gaussian Means/Variances

This section derives learning rules for the adaptation of the Gaussian kernels of the KSDG-SOM. These rules are in turn used in Section 5.4.1 that defines the update of weights and variances during the learning process in terms of them.

Clearly, formulas for the adaptation of both the Gaussian means and of variances should be derived. The accommodated Gaussian nodes with overlapping activation regions, instead of the usual SOM nodes with disjoint and uniform activation regions (Voronoi tesselation), increase significantly the potentiality of the model for effective learning of the input density [29]. Furthermore, a proper higher-dimensional feature space (e.g. the product space) can reveal interesting structure of the data (e.g. higher order statistics) which may go unnoticed in the standard Euclidean space [30].

Denote by L a lattice of N nodes and by $I \subset \Re^d$ the input space. Each node $i \in L$ is characterized by its weight vector $\mathbf{w}_i = [w_{i1} \dots w_{id}] \in I$ and by a lattice coordinate $\mathbf{r}_i \in L_A$, where L_A is the lattice space.

Instead of directly computing the activation of a node with weight vector $\mathbf{w}_i$ from the input vector $\mathbf{x}$ by the inner product $\langle \mathbf{x}, \mathbf{w}_i \rangle$, we utilize a kernel function $K(\mathbf{x}, \mathbf{w}_i) = \langle \Phi(\mathbf{x}), \Phi(\mathbf{w_i}) \rangle$. The selected kernel is the Gaussian one:

$$K(\mathbf{x}, \mathbf{w}_i, \sigma_i) = \exp(-\frac{\|\mathbf{x} - \mathbf{w_i}\|^2}{2\sigma_i^2})$$

Following the common practice of SOM-like algorithms, we want to minimize the distortion between the mapping of the input $\Phi(\mathbf{x})$, assigned to node i, and the mapping of the node $\Phi(\mathbf{w}_i)$, i.e. the prototype of node i in kernel space. Therefore we perform gradient descent with respect to $\mathbf{w}_i$ in order to adapt it properly:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}_i} \|\Phi(\mathbf{x}) - \Phi(\mathbf{w}_i)\|^2 &= \frac{\partial}{\partial \mathbf{w}_i} K(\mathbf{w}_i, \mathbf{w}_i, \sigma_i) + \frac{\partial}{\partial \mathbf{w}_i} K(\mathbf{x}, \mathbf{x}, \sigma_i) \\ &- 2\frac{\partial}{\partial \mathbf{w}_i} K(\mathbf{x}, \mathbf{w}_i, \sigma_i) \\ &= -2\frac{\partial}{\partial \mathbf{w}_i} \exp\left(-\frac{\|\mathbf{x} - \mathbf{w}_i\|^2}{2\sigma_i^2}\right) \end{aligned} \tag{1}$$

Thus the update rule for the kernel centers $\mathbf{w}_i$ should be of the following form:

$$\Delta \mathbf{w}_i = \mu_w \frac{(\mathbf{x} - \mathbf{w}_i)}{\sigma_i^2} K(\mathbf{x}, \mathbf{w}_i, \sigma_i) \tag{2}$$

with μ_w the learning rate for the kernel centers, referred as *kernel centers learning rate parameter* μ_w.

The next step is to derive the learning rule for the kernel radii σ_i, $\forall i \in L$. The parameters σ_i correspond to the variance with a Gaussian mixture model interpretation. An effective policy for their initialization is to set them equal to the median of the Euclidean distances from each positive training set member to the nearest negative. By performing gradient descent on $\|\Phi(\mathbf{x}) - \Phi(\mathbf{w}_i)\|^2$, with respect to σ_i, we obtain:

$$\Delta \sigma_i \propto \frac{\|\mathbf{x} - \mathbf{w}_i\|^2}{\sigma_i^3} K(\mathbf{x}, \mathbf{w}_i, \sigma_i)$$

Therefore the update rule for the Gaussian centers will be of the form:

$$\Delta\sigma_i = \mu_\sigma \frac{\|\mathbf{x} - \mathbf{w}_i\|^2}{\sigma_i^3} K(\mathbf{x}, \mathbf{w}_i, \sigma_i) \tag{3}$$

with μ_σ the learning rate for the kernel variances (i.e. radii), referred as *kernel variance learning rate parameter* μ_σ.

Typical values that we have used for the kernel centers learning rate parameter μ_w and the kernel variance learning rate parameters μ_σ, are $\mu_w = 0.1$ and $\mu_\sigma = 0.1$. However, the convergence of the algorithm is not sensitive to the exact values of these parameters. We obtained convergence even for large values of μ_w, μ_σ, i.e. $\mu_w = 0.5$, $\mu_\sigma = 0.7$. Clearly, for small values (e.g. $\mu_w = 0.01, \mu_\sigma = 0.01$) convergence is obtained, as it was expected.

4. The reclassification cost criterion

In this section we seek for a cost (error) function, the minimization of which will produce meaningful *reclassifications* of the original data set that account also the input variance (i.e. unsupervised part) and the model complexity. We proceed by embedding the problem of the construction of the new classes within the framework of *maximum likelihood*.

The term *gene expression profile* refers to the measurements of the differential expression levels of the genes observed under different induced conditions. It is represented with a vector of the measurement data values, and corresponds to a row of the gene expression matrix.

Denote by X the set of N gene expression profiles, i.e. $X = \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ and $L = \{l_1, \ldots, l_N\}$ their functional class labelings. We form a new *label augmented* set $Q = \{\mathbf{q}_1, \ldots, \mathbf{q}_N\}$ from the original gene expression profiles plus the extra functional class labeling attributes, i.e. $\mathbf{q}_i = (\mathbf{x}_i, l_i)$.

The labeling set L at the application domain of gene expression data analysis is usually either incomplete and/or unreliable. In order to confront to some extent these deficiencies we develop systematically a cost criterion for a reclassification approach. Reclassification aims to find *partitions* $\{C_1, \ldots, C_K\}$ of the label-augmented set Q. The number of the formed partitions K will be the number of the derived classes by the reclassification approach. Clearly, this number should be related to the number K_0 of the original functional labelings. An effective partition should combine elegantly the information contained both in X and L.

We proceed by formalizing reclassification from the point of view of discovering partitions of the label-augmented data set Q. Using a maximum likelihood formulation, we derive the equation for the error criterion that we should optimize subsequently with the KSDG-SOM learning algorithm.

Each partition of the data set Q is specified by the pairs $(\mathbf{q}, \mathbf{r})$, where $\mathbf{r}$ is the representative (i.e. prototype) associated with $\mathbf{q}$. The assignment of a representative $\mathbf{r}$ to the (label-augmented) pattern $\mathbf{q}$ is independent of the rest assignments.

Therefore, the likelihood L of a partition of the N label-augmented patterns of the set Q with representatives (i.e. prototypes) $(\mathbf{r}_i, \ldots, \mathbf{r}_K)$ is

$$L = \prod_{\mathbf{q} \in Q} \text{prob}(\mathbf{q}, \mathbf{r}) = \prod_{\mathbf{q} \in Q} \text{prob}(\mathbf{q}|\mathbf{r}) \cdot \text{prob}(\mathbf{r}) \tag{4}$$

A common practice is to minimize the negative logarithm of the likelihood

$$E = -\ln L = -\sum_{\mathbf{q} \in Q} \ln \text{prob}(\mathbf{q}|\mathbf{r}) - \sum_{\mathbf{q} \in Q} \ln \text{prob}(\mathbf{r}) \tag{5}$$

where E is the *error* function. The term $-\sum_{\mathbf{q} \in Q} \ln \text{prob}(\mathbf{r})$ of equation 5 represents the a priori probability for the partition and can be droped when we assume equal a priori probabilities for all the possible partitions. Though, in the absence of a priori knowledge, a sensible postulation is the new partitions to follow a gaussian distribution $N(K_0, \sigma_{K_0})$ around the number K_0 of the original classes, i.e.

$$\text{prob}(\mathbf{r}) = \frac{1}{\sqrt{(2\pi\sigma_{K_0}^2)}} \exp(-\frac{(K - K_0)^2}{2\sigma_{K_0}^2}) \tag{6}$$

Using equation 6 in 5, we obtain the following error criterion, which *explicitly penalizes the model complexity*:

$$E = -\sum_{\mathbf{q} \in Q} \ln \text{prob}(\mathbf{q}|\mathbf{r}) + \frac{N}{2\sigma_{K_0}^2}(K - K_0)^2 \tag{7}$$

We assume a *generative model* for the data generation process, in which each gene expression pattern $\mathbf{q}$ is derived from its cluster representative $\mathbf{r}$ by adding gaussian noise $\mathbf{n}$ of zero mean and variance σ^2, i.e. $\mathbf{n} = N(\mathbf{0}, \sigma)$,

$$\mathbf{q} = \mathbf{r} + \mathbf{n} \tag{8}$$

Therefore, the conditional probability of generating $\mathbf{q}$ given that the $\mathbf{r}$ representative is selected becomes,

$$\text{prob}(\mathbf{q}|\mathbf{r}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{(\mathbf{q} - \mathbf{r})^2}{2\sigma^2}) \tag{9}$$

Substituting equation 9 into 7 and ignoring the constant terms, we obtain the following sum-of-squares error function that incorporates also an extra term that controls the complexity of the partition $(C_1, \ldots, C_K)$ with representatives $(\mathbf{r}_1, \ldots, \mathbf{r}_K)$:

$$E = \frac{1}{2\sigma^2} \sum_{k=1}^{K} \sum_{\mathbf{q} \in C_k} (\mathbf{q} - \mathbf{r}_k)^2 + \frac{N}{2\sigma_{K_0}^2}(K - K_0)^2 \tag{10}$$

Recapitulating, on the rigid basis of the maximum likelihood approach, we have derived a sum-of-squares error function for reclassification that includes a model complexity penalty term. It is important to note that the first term of equation 10 evaluates both the unsupervised and the supervised part of the error. Also, at the squared difference of the first term of equation 10 the difference of functional

labels should account for the *within cluster functional discrepancy of the patterns.* Instead of using squared differences between functional assignments of the patterns and those of the class representations we considered more effective metrics to quantify this quantity with an entropy like measure (described in Section 5). Also, we will define the newly formed partition $(C_1, \ldots, C_K)$ in terms of the *superclusters* defined over the KSDG-SOM nodes, i.e. each $C_i,\ i = 1, \ldots, K$ will be a supercluster (superclusters are defined in Subsection 5.1). At this point we should stress that at the context of the KSDG-SOM we use the term *cluster* for the patterns allocated to the same KSDG-SOM node.

5. The KSDG-SOM algorithm

This section is organized in two parts: The first subsection elaborates further the general error criterion for optimization derived in Section 4 (i.e. equation 10). The next subsection details on the algorithms for the minimization of this error.

5.1. *Elaboration of the error criterion for optimization*

According to equation 10, the KSDG-SOM learning process should aim to minimize a heterogeneous [b] performance measure (i.e. error function). Denoting by K_n the number of KSDG-SOM nodes, which correspond (as noted previously) to *clusters* of patterns, clearly $K > K_n$ since each supercluster is composed of one or more clusters. Also, since the first term of equation 10 includes both unsupervised and supervised components, we can reformulate the general form of the error function in terms of SDG-SOM nodes and with distinct unsupervised and supervised parts as:

$$E = \sum_{k=1}^{K_n} \text{UnsupervisedComponent}_k + r_{su} \cdot \sum_{k=1}^{K_n} \text{SupervisedComponent}_k + \text{ModelOrderPenalty} \quad (11)$$

The UnsupervisedComponent will be measured in terms of the AverageLocalError(abbreviatedALE) and the SupervisedComponent by means of an entropy-like quantity (abbreviated EntropyLike) that generally is not an entropy with the strict definition. Also, r_{su} is a parameter that controls the *relative significance of the supervised part.* Restating of these quantities we obtain a measure Θ_E to minimize:

$$\Theta_E = \min \left(\sum_{k=1}^{K_n} ALE_k + r_{su} \cdot \text{EntropyLike}_k + MOP \right) \quad (12)$$

where ALE denotes the Average Local Error, EntropyLike is the forementioned entropy-like parameter, MOP abbreviates the Model Order Penalty and K_n is the number of nodes. This minimization is achieved with the formulation of SOM-like learning rules and with a dynamic expansion process.

[b] Heterogeneous in the sense that integrates supervised, unsupervised and model complexity terms

The parameter ALE of Equation 12 accounts for the unsupervised (quantization) error corresponding to the representation of each pattern i by the codebook of node k and can deal with the lack of class information. This measure tries to disperse patterns that are different according to some similarity metric, to different clusters, even if they are labeled with the same functional class label.

A commonly used measure for the Local Error (LE), and the one that we minimize with the formulation of equation 1, is the Euclidean distance between the kernel mapping $\Phi(\mathbf{x_i})$ of input vector $\mathbf{x_i}$ and the kernel mapping $\Phi(\mathbf{w}_k)$ of the representative prototype $\mathbf{w}_k$ of its best matching node k, i.e.

$$\mathrm{LE}_i = \|\Phi(\mathbf{x}_i) - \Phi(\mathbf{w}_k)\|^2 \tag{13}$$

Then the Average Local Error (ALE) is obtained by averaging the Local Error (LE) over all the patterns mapped to the same node. The justification for this averaging is explained in Section 6.

The available a priori information for the functional class of the patterns is considered by the *entropy like measure*. This measure corresponds to an entropy like quantity that characterizes the uniformity of functional labeling of the node where the pattern is mapped. It is formally defined in Section 6. The entropy measure is effective for evaluating category discordances over the discrete nominal category space of gene functional labelings. Minimization of this measure is performed by *gathering genes with the same (or at least similar) labels onto the same clusters*.

The MOP (ModelOrderPenalty) term punishes any increase in the model complexity. In this framework, the model complexity relates to the number K_s of superclusters of KSDG-SOM nodes.

The *superclusters* on the KSDG-SOM lattice (viewed as undirected graph) are defined as the *connected components* of the graph that *represent the same class labels*. It is clear that the superclusters can partition the state space in regions of *arbitrary shape* (i.e. not necessarily spherical as the case of the clusters corresponding to KSDG-SOM nodes).

Since superclusters define relatively homogenous regions of the state space, we found more appropriate to define the model complexity in terms of these. We define the *model order penalty* in terms of the squared difference between the number of the original classes and the number of developed superclusters that represent the newly formed (i.e. discovered) classes. We should note that very simple models consisted of less superclasses than the original classes are also punished.

Specifically, denoting (as usually) by K_0 the number of original classes and by K_s the number of superclusters, a term of the following form is exploited:

$$\mathrm{MOP} = \gamma \cdot (K_0 - K_s)^2 \tag{14}$$

In order to set properly the parameter γ we use a well working heuristic: we require that by doubling the number of original classes K_0 (i.e. $K_s = 2 \cdot K_0$) we get a penalty equal to the unsupervised + supervised part of the error. This criterion was found

appropriate to automatically set a value for the γ parameter at the KSDG-SOM implementation.

With $r_{su} = 0$ we have pure unsupervised learning with model complexity penalization. As r_{su} increases, the cost Θ_E is minimized for configurations that fit better to the a priori classification. Finally, for sufficiently large values of r_{su}, the a priori component dominates completely. Clearly, since in this case the information provided by the data is demolished, care should be taken to avoid such r_{su} values.

After the former discussion on the error components that the KSDG-SOM aims to jointly minimize, we can now proceed to describe the corresponding learning algorithms.

5.2. *The KSDG-SOM dynamic growing and adaptation*

The KSDG-SOM is initialized with four nodes arranged in a 2X2 rectangular grid and grows nodes to represent the input data. This type of initialization is somehow arbitrary and different starting configurations can be used, e.g. 9 nodes arranged as a 3X3 grid. Weight values of the nodes are self-organized according to a new method inspired by the SOM algorithm. The self-organization process maps properties of the original high-dimensional data space onto the lattice consisted of KSDG-SOM nodes. The map is expanded to represent the input space by creating new nodes, either from the boundary nodes performing *boundary extension*, or by inserting whole columns (or rows) of new units with a *column extension* (or row extension).

A *training epoch* consists of the presentation of all the training gene expression patterns to the KSDG-SOM. A *training run* is defined as the training of the KSDG-SOM with a fixed number of neurons at its lattice i.e. the training between successive node insertions/deletions.

The top-level KSDG-SOM learning in algorithmic notation can be described as:

<Top-level KSDG-SOM learning algorithm>

1. Initialization (sets $r_{su} = 0$, i.e. pure unsupervised learning). (Subsection 5.3)
Repeat // develop a series of models corresponding to increasing
    // consideration of the supervised parameter, r_{su}
    Repeat
        2. Training Run Adaptation phase. (Subsection 5.4)
        3. Expansion phase (Section 6)
    until criteria for stopping map expansion are satisfied (Section 7)
4. Fine Tuning Adaptation phase (Subsection 5.6)
5. Save configuration of the map for the current supervised/unsupervised ratio, r_{su}.
6. Compute classification performance for the current r_{su}
7. Increment the significance of the supervised part, i.e. increase ratio r_{su}
until *Classification Performance* $\approx$ *1*
8. Model Selection Step (Subsection 5.8)

The details of the algorithm, i.e. the initialization, adaptation, fine tuning phases and the corresponding convergence criteria are described in detail below. The technical subleties involved in the expansion process are scrutinized detail in section 6.

5.3. *Initialization phase.*

The weight vectors of the four starting nodes that are arranged in a 2X2 grid are initialized with random numbers within the domain of feature values. Other initialization schemes are possible i.e. as it has been noted we can initialize to a 3X3 grid. The supervision parameter r_{su} controls the tradeof between unsupervised and supervised training and is discussed in detail in Section 6. It is initialized to 0, i.e. pure unsupervised learning is performed for the first KSDG-SOM model being generated.

5.4. *Training Run Adaptation phase.*

The purpose of this phase is to stabilize the current map configuration in order to be able to evaluate its effectiveness and the requirements for further expansion. During this phase, the input patterns are repeatedly presented and the corresponding self-organization actions are performed until the map converges sufficiently. The training run adaptation phase takes the following algorithmic form.

<Training Run Adaptation Phase>:
MapConverged := **false**;
while *MapConverged* = **false do**
  for all input patterns **do**
    present and adapt the map by applying the *map adaptation rules* (Subsection 5.4.1)
  endfor
    Evaluate map *training run convergence condition* (Subsection 6.1)
    and set MapConverged accordingly
endwhile

The map adaptation rules are described below while the training run convergence condition is described in Section 6.1, after the appropriate terms have been defined.

5.4.1. *Map adaptation rules*

The map adaptation rules that govern the processing of each input pattern $\mathbf{x}_k$ are as follows:

(1) Determination of the weight vector $\mathbf{w}_i$ for which its kernel mapping $\Phi(\mathbf{w}_i)$ is closest to the kernel mapping $\Phi(\mathbf{x}_k)$, of the input vector $\mathbf{x}_k$, according to the utilized distance measure (i.e. determination of the winner node).

(2) Adaptation of the weight vectors (i.e. Gaussian centers) only for the four nodes in the direct neighborhood of the winner and for the winner itself according to the following formula:

$$\mathbf{w}_j(k+1) = \begin{cases} \mathbf{w}_j(k), & j \notin N_k \\ \mathbf{w}_j(k) + \mu_w \cdot \Lambda_k(d(i,j)) \cdot \Delta\mathbf{w}_j(k), & j \in N_k \end{cases} \tag{15}$$

(3) Adaptation of the Gaussian spreads also only for the four nodes in the direct neighborhood of the winner and for the winner itself according to the following formula:

$$\sigma_j(k+1) = \begin{cases} \sigma_j(k), & j \notin N_k \\ \sigma_j(k) + \mu_\sigma \cdot \Lambda_k(d(i,j)) \cdot \Delta\sigma_j(k), & j \in N_k \end{cases} \tag{16}$$

where the learning rates $\mu_w(k),\ \mu_\sigma(k),\ k \in N$, are monotonically decreasing sequence of positive parameters, N_k is the neighborhood of the winner node at the kth learning step and $\Lambda_k(d(j,i))$ is the neighborhood function implementing different adaptation rates even within the same neighborhood. Also, the gradient information $\Delta\mathbf{w}_j(k)$, $\Delta\sigma_j(k)$ for the weight and variance adaptation, was derived in Section 3 (i.e. equations 2, 3). Clearly, both parameters are defined in terms of the kernel distance metric.

The learning rates $\mu_w(k),\ \mu_\sigma(k),\ k \in N$ typically start from a value of 0.1 and decrease down to 0.02. These values are specified with the empirical criterion of having relatively fast convergence, without however sacrificing the stability of the map.

The KSDG-SOM starts with a much smaller size than a usual SOM. Therefore a large neighborhood is not required to train the whole map at the first learning steps (e.g. with 4 nodes initially at the map, a neighborhood of 1 only is required). As training proceeds, during subsequent training epochs, the area defined by the neighborhood becomes localized near the winning neuron, not by shrinking the vicinity radius (as in the standard SOM) but by enlarging the SOM with the dynamic growing.

The neighborhood function $\Lambda_k(d(j,i))$, can thus be defined with the following simple formula (the row and column of a node i are denoted by i_r, i_c respectively):

$$\Lambda_k(d(j,i)) = \begin{cases} 1 & \text{if } j = i \\ \alpha,\ 0 < \alpha < 1, & \text{if } |i_r - j_r| + |i_c - j_c| = 1 \\ 0, & \text{otherwise} \end{cases}$$

5.5. *Expansion Phase*

This phase constitutes the main core of the learning algorithms. It is described in detail in section 6.

5.6. *Fine Tuning Adaptation Phase*

The fine tuning phase aims to optimize the final KSDG-SOM configuration. This phase is similar to the training run adaptation phase described previously (subsec-

tion 5.4) with two differences:

(1) The final criterion for map convergence is more elaborated. We require much smaller change of the Total Growth Parameter for accepting the condition for map convergence.
(2) The learning rate decreases to a smaller value in order to allow fine adjustments to the final structure of the map.

Typically, the *ConvergenceErrorThreshold* for the fine tuning phase is about 0.00001 and the learning rate is set to 0.01 (or to an even smaller value).

5.7. *Evaluation of classification performances*

To each KSDG-SOM node is assigned a *classification vector* **CL** with elements $CL_l = pr_l$, where pr_l is the ratio of the gene expression patterns with functional label l, among all the gene expression patterns mapped to the node. This vector is considered as the predicted classification of new gene expression patterns that are evaluated on a trained KSDG-SOM device.

This classification is a soft one: each CL_l expresses the probability that to a node (and consequently the mapped patterns) is assigned a label l. We therefore compute the performance based on a metric proposed in [31].

Specifically, for each class label l of each pattern i, a score $sc_{l,i}$ is assigned. This score equals to the classification vector element for class label l, CL_l, formely defined as $CL_l = pr_l$, if the corresponding label is included in the original class assignment (i.e. $c_{l,i} = 1$) and equals $qr_l = 1 - CL_l$ in the other case (i.e. $c_{l,i} = 0$). In essence the score $sc_{l,i}$ measures the *consensus of the functional labeling* for class l between the pattern i and the node. Clearly, by summing over all labels we characterize with the TotalScore_i parameter, the consensus of the functional labeling between the pattern i and the node it is mapped onto. This parameter is calculated as:

$$\text{TotalScore}_i = \sum_{l=1}^{N_c} sc_{l,i}, \text{where } sc_{l,i} = \begin{cases} CL_l & \text{if } c_{l,i} = 1 \\ 1 - CL_l & \text{if } c_{l,i} = 0 \end{cases}$$

Intuitively, a small $CL_l \approx 0$ for a class that does not appear as a functional label (i.e. $c_{l,i} = 0$) for an input pattern is much more a success than a failure, therefore being considered by a score $1 - CL_i \approx 1$.

The performance for each pattern i, perf_i is then obtained by dividing this score with the total number of functional class labelings, N_c, i.e.

$$\text{Perf}_\text{i} = \frac{\text{TotalScore}_i}{N_c}$$

The global measure of the performance *ClassificationPerformance*(r_{su}) for a given ratio r_{su} (i.e. the supervision weighting parameter of equation 12), is obtained by averaging the Perf_i values for all the gene expression patterns of the

testing set, TS i.e.

$$\text{ClassificationPerformance}(r_{su}) = \frac{\sum_{i \in TS} \text{Perf}_i}{|TS|}$$

where the notation $|TS|$ denotes the number of elements of the testing set. The testing set was obtained by splitting the original data sets into ten approximately equal parts and using the *tenfold cross validation* methodology to evaluate the performance [32].

5.8. *Model Selection Step*

During this step a well performing ratio r_{su} is selected by using the following criteria:

- The classification performance has obtained significant increase for the selected r_{su} parameter value at the corresponding classification performance curve and this increase is followed by a plateau. The increase at the classification performance with increasing r_{su}, means that a priori information for the application was taken into account by the second term of Equation 12 that enforces class labeling uniformity at the formation of the cluster boundaries. The plateau that we require to follow the steep increase implies that increasing further the strength of the a priori information, although it can bias the model heavily towards an imperfect domain theory, does not offer significant improvements to its generalization potential. This methodology for model selection will be illustrated better by means of an example in Section 8.
- The number of the KSDG-SOM nodes that grow for the "optimal" r_{su} value should be relatively small (small model complexity). This criterion prefers the models with the smallest complexity that offer adequate generalization performance.

We should note that these selection criteria are somewhat heuristic. However, there seem to perform an adequate model selection.

5.9. *Node Deletion*

Nodes that are selected as winners for very few (usually one or two) training patterns, termed *uncolonized* nodes, are not deleted by our scheme although they probably correspond to noisy outliers. The gene expression patterns that consistently (three times or more) are mapped to uncolonized nodes are very unique and can either be artifacts or if not they have the potential to provide knowledge. Therefore they are amenable to further consideration. These patterns therefore are marked and isolated for further study. Nodes that are not selected as winners for any pattern are removed from the map in order to keep it compact.

6. The Expansion Process

The expansion is based on the detection of the neurons with large *Growth Parameter* (GP), referred to as the *unresolved neurons*. The node with the largest GP becomes the current focus of map expansion.

The Growth Parameter for node i, denoted GP_i, is based on an heterogeneous type of error, computed as:

$$GP_i = ALE_i + r_{su} \cdot \text{EntropyLike}_i \tag{17}$$

where we recall that the ALE denotes the Average Local Error.

We describe in turn the two components of 17, i.e. the *average local error* and the *entropy*.

A *local error* term is commonly used for implementing dynamically growing schemes [33,9]. A general assessment of the local error, le_i, is given by

$$le_i = \sum_{\mathbf{x} \in S_i} \text{Dist}(\Phi(\mathbf{x}), \Phi(\mathbf{w}_i)) \tag{18}$$

where we denote by S_i the set of gene expression patterns $\mathbf{x}$ mapped to node i, $\mathbf{w}_i$ the weight vector of node i that corresponds to the average expression profile of S_i and the Dist operator denotes the corresponding distance metric.

However, the peculiarities of the gene expression data analysis application domain motivated two significant modifications to the classic local error measure. Specifically:

(1) Instead of the simple local error measure of Equation 18 we use the *average local error* ALE_i per pattern, defined as:

$$ALE_i = \frac{le_i}{|S_i|} \tag{19}$$

where $|S_i|$ denotes the number of elements of the set $|S_i|$.

This measure does not increase when many similar gene expression patterns are mapped to the same node.

(2) The second provision applies when we have class information available (either complete or partial) and we want to exploit it in order to improve the expansion. The local error that accumulates to a winner node is amplified by a factor that is inversely proportional to the square root of the frequency ratio r_c of its corresponding class c. Specifically, let $r_c = \frac{\#\text{patterns of class } c}{\#\text{total patterns}}$ be the frequency ratio of class c. Then the amplification factor is $r_c^{-\frac{1}{2}}$.

The supervised contribution to the heterogeneous error of Equation 12 is based on the computation of a parameter HN_i characterizing the *entropy of the class assignment content* of each node i. An advantage of the entropy is that it is relatively insensitive to the over-representation of classes. This means that independently of how many gene expression patterns of a class are mapped to the same node, if the node does not represent significantly other classes, its entropy is very small.

We first consider the simple case of each pattern belonging only to one functional class. The assignment of a class label to each neuron of the KSDG-SOM is in this case performed according to a majority-voting scheme [34]. The *entropy* parameter, that quantifies the uncertainty of the class label of neuron m, can be directly evaluated by counting the votes at each SOM neuron for every class as [35]:

$$HN(m) = -\sum_{k=1}^{N_c} p_k \cdot \log p_k \tag{20}$$

where N_c denotes the number of classes and $p_k = \frac{V_k}{V_{pattern}}$, is the ratio of votes V_k for class k to the total number of patterns $V_{pattern}$ that vote to neuron m. The parameter p_k has a probability interpretation, i.e. it is computed as the relative frequency of votes for each class k. For the single label case, the number of labeled gene expression patterns $V_{pattern}$ is also equal to the number of votes.

Clearly, the entropy $HN(m)$ is zero for unambiguous neurons and increases as the uncertainty about the class label of the neuron m increases. The upper bound of $HN(m)$ is $\log(N_c)$, and corresponds to the situation where all the classes are equiprobable (i.e. the voting mechanism does not favor a particular class).

For the multi-label case, the voting scheme remains the same, but each pattern in this case can vote for more than one class. The frequent case of having gene expression patterns not assigned to any functional class is handled as a vote to the *Unassigned* class.

A quantity $HR(m)$ is defined similarly:

$$HR(m) = -\sum_{k=1}^{N_c} r_k \cdot \log r_k \tag{21}$$

The r_k do not correspond to probabilities but they are class voting ratios defined as the p_k (i.e. $r_k = \frac{V_k}{V_{pattern}}$). However, in this case $\sum_k V_k > V_{pattern}$ and therefore $\sum_{k=1}^{N_c} r_k > 1$. Thus, $HR(m)$ is not mathematically an entropy of a probability distribution. However, this quantity retains properties similar to the entropy.

We consider an example in order to explain the handling of multiple labeling by equation 21. Let $N_c = 3$ and suppose that 30 gene expression patterns are assigned to node m_1, all of them having as a label all the three classes and that 90 patterns are assigned to neuron m_2 one third of them having as a label class 1, another third class 2 and the last third class 3. Although in each case there are 30 patterns voting for each class, the quantity HR will be high in the case of the 90 patterns (i.e. $\log(3)$) and zero at the other case. Thus, the HR measure has quantified effectively the similarity of multiple class labeling between patterns of some cluster.

Since the expansion process involves a lot of algorithmic/programmatic details we present it in Appendix A.

6.1. *Evaluation of the map training run convergence condition*

The Total Growth Parameter "(TGP) is defined by summing the Growth Parameter (defined with equation 17) of all nodes, i.e.

$$TGP = \sum_{i \in \text{KSDGSOMnodes}} GP_i$$

.

The reduction of the *Total Growth Parameter* (TGP, defined in section 6) controls the training run convergence condition. The corresponding convergence test is:

$$\text{MapConverged} := \left(\frac{|TGP_b - TGP_a|}{TGP_a} < \text{ConvergenceErrorThreshold} \right)$$

where $TGP_b = \sum_i (GP_i)_b$ and $TGP_a = \sum_i (GP_i)_a$ denote respectively the sum of the Growth Parameters for all nodes before and after the presentation of patterns (i.e. one training epoch) and the *ConvergenceErrorThreshold* is a given value.

The above formula states that the map converges when the relative change of the TGP parameter between successive epochs drops below the threshold value. The setting of the *ConvergenceErrorThreshold* is somewhat empirical but a value in the range 0.01 - 0.02 performs well in assuming sufficient convergence without excessive computation.

7. Criteria for controlling the KSDG-SOM dynamic growing

The growing process of the KSDG-SOM is controlled by a Boolean criterion that checks for remaining random like clusters (at the algorithmic description of the Appendix A corresponds to the boolean function *RandomLikeClustersRemain*). Ideally, the output of this function should remain true until automatically the appropriate level of expansion is reached.

Intuitively, the growing should stop when the gene expression patterns are characterized by inter pattern distances that deviate significantly (either much smaller or much larger) from the typical distances for randomly selected patterns. The problem can be concentrated to the definition of similarity (or alternatively randomness) between patterns and to the determination of the maximum percentage of "random" gene expression patterns allowed to be allocated to the cluster.

The measure of distance is used to quantify the degree of similarity between pairs of gene expression patterns. At the case where the random distribution of the values of the distances were known, a one tail test could be applied. In this case, a confidence level α_l, can be defined for a given distance threshold. This confidence level means that distances equal or smaller the threshold value are found only in the proportion of α_l when accessing unrelated (i.e. "random") genes.

Although the true random distribution of the distances is uknown, it can be approximated by resampling the original data set. For each gene expression pattern, we shuffle randomly all its points (corresponding as usually to the columns, i.e.

experimental conditions, of the gene expression matrix). This reshuffling destroys the actual correlation between the different patterns, while at the same time retains the other characteristics of the data set (i.e. range and frequency of values, number of points). In this way, we compute an approximation to the distribution of the distance between random patterns.

In order to treat the problem quantitatively, we use a *randomization ratio* parameter α that corresponds to the aforementioned confidence level α_l parameter for the idealized random distribution. The randomization ratio controls how much of the randomized distances can be treated as non-random.

The randomization ratio α has the meaning that the probability that two patterns are "random" (unrelated) is lower than α, if the distance between them is either smaller than the threshold dL_{thr} or larger than dU_{thr}. From the randomization ratio α we derive distance thresholds dL_{thr} and dU_{thr}. This derivation is performed by constructing a histogram of the distribution of the randomized distances after the reshuffling. Then we sort the computed distances by increasing distance values. The threshold dL_{thr} (dU_{thr}) corresponds to the lower (upper) "tail" of the distribution and can be estimated as the distance corresponding to the $\frac{\alpha}{2} \cdot N_{patterns}$ ($(1-\frac{\alpha}{2}) \cdot N_{patterns}$) element of the sorted array, where $N_{patterns}$ denotes the number of patterns participated at the randomization experiment (we used all the available patterns).

Distances below dL_{thr} imply *positive correlation* between the expression patterns of the genes. In a similar manner, distances larger than dU_{thr} imply *negative correlation*. Both cases imply non-random association between the genes, i.e. of coregulation of their expression levels.

Figure 1 illustrates the distribution of the distances (with the Manhattan distance metric) between the random gene expression patterns and the actual patterns from a gene expression data experiment. Specifically, the experiment analyzed concerns the 6,221 genes from the budding yeast Saccharromyces cerevisiae public available from the Stanford web site and functionally labeled as described in Section 8.

In this case, we can determine an upper dU_{thr} and a lower dL_{thr} distance threshold and consider the distances lying in the interval $[dL_{thr}\ dU_{thr}]$ as random.

Having computed the distance thresholds dL_{thr}, dH_{thr} we can control automatically the KSDG-SOM dynamic growing process with only one parameter, the $p_{decompose}$ parameter. The $p_{decompose}$ parameter controls the percentage of inter-node "random" pattern distances, i.e. patterns x_i, x_j with distances $dL_{thr} < \text{dist}(x_i, x_j) < dU_{thr}$, allowed before expansion is initiated. The parameter $p_{decompose}$ is specified empirically to a value of 2%. Clearly, the smallest the parameter $p_{decompose}$ the more the KSDG-SOM is splitted.

In conclusion, given the values of the randomization ratio α and the percentage $p_{decompose}$, the growing process is controlled by taking into account the distribution of the data as illustrated in figure 1, and as a result an appropriate number of

clusters is determined automatically.

8. Results and Discussion

This section first describes briefly the characteristics of the particular DNA microarray data analyzed. Then we proceed with the application of the KSDG-SOM for the analysis of these data. We evaluate the KSDG-SOM technique comparatively with other competent machine learning approaches. Also we artifically disturb the functional classifications in order to assess the strength of the model as a reclassification device.

We have applied the KSDG-SOM to analyze microarray expression data from the budding yeast Saccharomyces cerevisiae. We explored two (related) data sets, one large, with expression profiles for every yeast gene and one smaller that is a "classic" dataset on which many experiments have been conducted. These data are public available from the Stanford web site. They were generated by studying this fully sequenced organism with microarrays, containing essentially every Open Reading Frame (ORF). The samples used were collected at various time points during the diauxic shift, the mitotic cell division cycle and sporulation. The larger data set consists of 80-element gene expression vectors for 6,221 genes and the smaller one includes 79-element gene expression vectors for 2,476 genes.

Specifically we have conducted the following experiments with these large datasets.

8.1. *Data Set 1*

The source of these profiles were eight different microarray experiments under different conditions. These conditions can be categorized into the following four types: 1. the mitotic cell division cycle, 2. sporulation, 3. temperature and reducing socks, 4. gene expression in the budding yeast during the diauxic shift.

For example, data for the last condition were obtained from [36]. With a fluorescence-ratio method, Derisi et. al. [36] measured the relative abundance of mRNA for the entire yeast genome, in yeast growing in a fresh medium to examine the changes in expression that take place with the metabolic shift from anaerobic to aerobic metabolism, with seven samples taken at 2-hour intervals. Measured levels of expression of genes in this experiment reflect metabolic reprogramming that occured during the diauxic shift.

Annotation for these genes was derived from the Functional Classification Catalogue of the Munich information center for protein sequences (MIPS) Comprehensive Yeast Genome Database (CYGD) available at http://mips.gsf.de/proj/yeast/CYGD/db/index.html The selected annotations included 5 from the 19 *top-level functional categories*. A total of 1974 genes belong to these categories. The decision to evaluate the algorithms on a subset of the whole large set, is motivating for obtaining better control of the evaluation process and from practical reasons that concern the large computational requirements of the

evaluated algorithms. Specifically, we selected the genes of the following categories for our data set:

(1) Cell Fate (423 ORFs)
(2) Cell Rescue, Defense And Virulence (273 ORFs)
(3) Cellular Communication/Signal Transduction Mechanism (59 ORFs)
(4) Cellular Transport And Transport Mechanisms (480 ORFs)
(5) Metabolism (1059 ORFs)

The gene expression data is arranged in a table whose rows correspond to the genes and columns to the individual log-transformed gene expression values of each gene in a particular experimental condition represented by the column. The weighted K-nearest neighbors imputation method presented in [37] is applied in order to fill up systematically the missing values. This data imputation approach detects the K most similar genes in expression to the one with missing values and estimates them by weighting the values of these genes at the same columns with their similarity.

The KSDG-SOM selects for the particular data set the model with a supervision parameter value of $r_{su} \approx 8$ as an appropriate model since as illustrated in Figure 2 it corresponds to a performance plateau. Detailed results are presented with Table 1. This selection of the model is performed according to heuristic criteria. We require acceptable classification performance, small number of nodes (i.e. model complexity control) and a plateau at the corresponding curve (i.e. small rate of increase of classification performance) that indicates that further increase at the supervised gain (i.e. of parameter r_{su}) should not be performed.

Also, Figure 4 illustrates a snapshot of the KSDG-SOM training. The grid on which the map grows is displayed continuously, in order to visually observe the learning process. Colors are used to denote the prevailing class of each node. Also, many other parameters are displayed in order to facilitate the monitoring and control of the learning process.

Reclassification Experiments

The reclassification experiments were performed by inducing randomly a small number of errors at the original labeling and by asking whether and to which extent the unsupervised drive can recover the original labeling. The revision of the incorrect supervised a-priori label is performed with the unsupervised force by assigning the pattern to a node of the correct class.

These experiments aim to evaluate the reclassification performance of KSDG-SOM. Specifically, we take as an example the Cell Fate class. We alter the labeling of some (usually 10 to 50) of the 423 Cell Fate genes and we ask for the extent in which the KSDG-SOM succeds in recovering the original classification by means of the unsupervised labeling information. We repeated the experiment on random partitions of the set in order to obtain statistical confidence (the reported values are averages of 20 repetitions). At this experiment we have observed that small r_{su} parameters, can recover a lot of the original labeling, in contrast to the large ones

r_{su}	Number of nodes K, Number of superclusters	Classification Performance(r_{su})
0	62, 13	0.19
1	65, 13	0.22
2	80, 12	0.23
3	92, 10	0.31
4	90, 12	0.38
5	63, 11	0.41
6	81, 10	0.44
7	95, 12	0.51
8	89, 13	0.68
9	79, 11	0.70
10	81, 12	0.72
11	91, 11	0.75
12	63, 10	0.78
13	80, 9	0.82
14	80, 9	0.83
15	82, 8	0.91
16	80, 8	0.92
17	86, 8	0.93
18	90, 7	0.94
19	89, 8	0.95
20	98, 8	0.965
21	92, 8	0.97
22	91, 8	0.98
23	94, 9	0.98
24	90, 7	0.98

Table 1. The model selection process for the KSDG-SOM model.

that enforce the incorrect labeling. We repeated this type of experiment and for other four classes. Table 2 presents the results obtained.

Although, the KSDG-SOM is not designed for the optimization of the supervised classification performance, it can obtain relatively affordable results and as a classification tool. However, we stress that the main aim of the KSDG-SOM is the reclassification task and its potentiality for the revision of a-priori knowledge. Table 3 presents results from the comparative evaluation of KSDG-SOM as a classification device with Data Set 1. We observe that the KSDG-SOM obtains better classification performance than the linear SDG-SOM (i.e. a similar algorithm without the kernel mapping). The Support Vector Machine obtains clearly the best supervised performance, although the KSDG-SOM can compete other less sophisticated supervised algorithms. All the algorithms were tested with the 10-fold cross

r_{su}	Cell Fate (427)	Cell Rescue (278)	Cellular Communication (59)	Cellular Transport (495)	Metabolism (1065)
0	0.91	0.8	0.82	0.81	0.8
2	0.9	0.75	0.77	0.76	0.74
4	0.9	0.72	0.7	0.71	0.72
6	0.8	0.6	0.6	0.62	0.59
8	0.6	0.4	0.5	0.48	0.49
10	0.5	0.35	0.25	0.26	0.27
12	0.4	0.31	0.26	0.27	0.25

Table 2. Results from reclassification experiments with the KSDG-SOM model from Data Set 1. The *class recovery performances* that the table presents indicate the percentage of successful "correction" of the induced classification errors by the unsupervised drive.

validation technique. The other algorithms are available from the WEKA machine learning Web site (http://www.cs.waikato.ac.nz/ml/weka/) and are implemented in the Java programming language [32]. The KSDG-SOM implementation is also in Java, so some comments on the running speed can apply more objectively. Specifically, our algorithm is faster than the implementation of the Support Vector Machine supported by WEKA, but not as fast as the other simpler methods. For the particular data sets, that consist of about 2000 values per gene the KSDG-SOM is about 30 to 50 (depending on parameter settings) times faster than the SVM implementation of WEKA.

Furthermore, we have observed that the rsu parameter affects significantly the classification performance of KSDG-SOM. Clearly, large rsu values favor the supervised bias and lead to significantly better classification performances at the expense of the potentiality for knowledge revision. The results of Table 3 illustrate that the classification performance of KSDG-SOM improves to some extent with a considerable increase of the supervised strength (i.e. from $rsu = 1$ to $rsu = 10$).

Clustering Performance Evaluation

Although (as we already stated) the main purpose of the KSDG-SOM development is not to serve as another one clustering device, the SOM-like clustering that performs at the kernel space, is a powerful basis for clustering. Indeed the KSDG-SOM can do better than other clustering approaches. It is well known that it is difficult to obtain generally applicable criteria for evaluating the quality of clustering algorithms. Thus we emphasize on the aspect of the *consistency* of the clustering results; clearly a very important issue for obtaining reliable solutions. We evaluate the KSDG-SOM from this point of view comparatively with two other widely used clustering algorithms.

The validation approach that we have used for evaluating the clustering performance of KSDG-SOM aims to reward the consistency of the results. We denote (as previously) by d the dimensionality of the gene expression vectors, i.e. the number

Class	Method	TP	TN	FP	FN	Precision	Recall
Cell Fate	KSDG-SOM (rsu=1)	289	1354	197	134	0.5947	0.6832
	KSDG-SOM (rsu=10)	312	1375	176	111	0.6393	0.7376
	SDG-SOM (linear)	243	1256	295	180	0.4517	0.5745
	Kernel Density	151	1331	220	272	0.407	0.357
	Naive Bayes	182	1327	224	241	0.448	0.43
	SVM	353	1490	4	127	0.984	0.735
Cell Rescue	KSDG-SOM (rsu=1)	130	1520	181	143	0.418	0.4762
	KSDG-SOM (rsu=10)	142	1579	122	131	0.5379	0.5201
	SDG-SOM (linear)	117	1428	273	156	0.3	0.42
	Kernel Density	69	1505	196	204	0.26	0.253
	Naive Bayes	77	1383	318	196	0.195	0.282
	SVM	196	1638	63	77	0.757	0.718
Cellular Communication	KSDG-SOM (rsu = 1)	11	1840	75	48	0.127	0.1864
	KSDG-SOM (rsu = 10)	18	1867	48	41	0.2727	0.3051
	SDG-SOM (linear)	10	1798	117	49	0.078	0.1695
	Kernel Density	2	1869	46	57	0.042	0.034
	Naive Bayes	37	1678	237	22	0.135	0.627
	SVM	9	1915	0	50	1	0.153
Cellular Transport	KSDG-SOM (rsu=1)	390	817	677	90	0.365	0.8125
	KSDG-SOM (rsu=10)	408	884	610	72	0.40008	0.85
	SDG-SOM (linear)	365	809	685	115	0.3476	0.7604
	Kernel Density	328	758	736	152	0.3083	0.6833
	Naive Bayes	340	807	687	140	0.331	0.708
	SVM	353	1490	4	127	0.989	0.735

Table 3. Results from the comparative evaluation of KSDG-SOM as a classification device with Data Set 1. We observe that the KSDG-SOM obtains better classification performance than the linear SDG-SOM (i.e. a similar algorithm without the kernel mapping). The SVM obtains clearly the best supervised performance, although the KSDG-SOM can compete other less sophisticated supervised algorithms.

of measurements that constitute the columns of the gene expression matrix. In order to evaluate the robustness of the KSDG-SOM clustering we exclude each time one column (i.e. one experiment) and we evaluate the clustering on the rest.

We require genes that were assigned to the same cluster (i.e. co-clustered) at the clustering result computed over the whole data set, to co-cluster and after the disturbance of excluding one coordinate (i.e. experiment). We define an NXN matrix D_0 with elements $D_0(i,j) = 1$ if genes i and j are co-clustered at the whole data set clustering and $D_0(i,j) = 0$ otherwise. Similarly, for each of the disturbed clusterings obtained by excluding experiment k, $k = 1, \ldots, d$, we define the matrices $D_k(i,j)$.

An effective criterion $R_{all}(N_c)$, for evaluating the consistency of the clustering algorithms can be obtained by seeking for the maximization of the quantity $R_{all}(N_c)$, defined as:

$$R_{all}(N_c) = \sum_{k=1}^{d} R(N_c, k), \quad R(N_c, k) = \frac{2}{N \cdot (N-1)} \sum_{j=1}^{i-1} \sum_{i=1}^{N} (1 - |D_0(i,j) - D_k(i,j)|) \quad (22)$$

that depends implicitly on the number of clusters N_c that each algorithm produces (normally decreasing with increasing the number of clusters).

We refer to the quantity $R_{all}(N_c)$ of equation 22 as the *average consistency ratio*. Intuitively, equation 22 counts the number of times that clusterings D_0 and D_k agree on whether or not to co-cluster each possible pair of genes. Also, we should note that when both clusterings D_0 and D_k do not co-cluster genes i and j equation 22 increases the consistency estimate as if they were co-clustered.[c]

Figure 3 illustrates the results from the evaluation of the average consistency ratio measure for the KSDG-SOM compared with two well-known clustering approaches, the K-Means and the Expectation Maximization. All algorithms have been evaluated for both the Euclidean and the Pearson correlation based distance metric. Therefore we obtain six curves that demonstrate the variation of the average consistency ratio for each approach with the number of clusters. We should note, that for the KSDG-SOM case the clusters correspond to superclusters as they were defined formely, and not to KSDG-SOM nodes. Also, the source code implementations of the K-Means and Expectation Maximization clustering packages (in the Java programming language) was obtained and adapted from the WEKA machine learning Web site (http://www.cs.waikato.ac.nz/ml/weka/) [32].

Referring to the results displayed by Figure 3, the dotted curve with star marks demonstrates the performance of one of the simpler clustering approches the K-Means algorithm with the Euclidean distance metric. This approach displays relatively consistent results for $N_c = 8$ and $N_c = 9$ but its performance drops significantly with increasing number of clusters. The K-Means with a distance metric based on the Pearson correlation coefficient displays improved performance (dash-dotted curve with circle markers). The Expectation Maximization approach outperforms from the point of view of the consistency of the obtained results the K-Means. Again the Pearson correlation distance metric (dotted curve with up-triangle markers) produces better results than the Euclidean metric (dotted curve with square markers).

The KSDG-SOM outperforms clearly the former clustering approaches in terms of the consistency of the results. Intuitively, the solution of the clustering problem in the kernel space, where the Gaussian basis functions implement smoother cluster boundaries, can be the theoretical justification for the consistency of the results (al-

[c]A possible extension/elaboration of this consistency evaluation scheme is to assign a larger weight to the co-clustered in both clusterings gene pairs over those gene pairs that are dispersed in both clusterings.

1	Tricarboxylic-acid pathway
2	Respiration-chain complexes
3	Cytoplasmic ribosomal proteins
4	Proteasome
5	Histones
6	Helix-turn-helix

Table 4. Functional classes used for supervised KSDG-SOM training. The tricarboxylic-acid pathway is, also known as Krebs cycle, consists of genes that encode enzymes that break down pyruvate (produced from glucose) by oxidation. The respiration chain complexes perform oxidation-reduction reactions that capture the energy present in NADH through electron transport and the chemiosmotic synthesis of ATP. The cytoplasmic ribosomal proteins are a class of proteins required to make the ribosome. The proteasome consists of proteins that perform the degradation of proteins. Histones interact with the DNA backbone to form nucleosomes. These nucleosomes are an essential part of the chromatin of the cell. Finally, the helix-turn-helix class, is not a functional class. It consists of genes that code for proteins containing the helix-turn-helix structural motif. This class is included as a control class.

though we do not elaborated on the rather difficult issue of analyzing theoretically the reasons of the consistency). Again, the distance metric obtained from the Pearson correlation coefficient, exceeds in performance the Euclidean distance metric. All evaluations of the KSDG-SOM for clustering were obtained by setting $rsu = 0$ for pure unsupervised operation.

8.1.1. *Data Set 2*

Another experiment that we have performed is aimed to evaluate the supervised classification performance and consists of training the KSDG-SOM with temporal expression patterns of 2,476 genes of the yeast measured in eight experimental settings (available online at http://rana.stanford.edu/clustering/). Some details on the data can be found in [4]. As usual, each sample measures the expression level of a gene compared to the expression level in a reference state. Totally, there were 79 time points for each gene. We used as a test base the same functional classes as in [11]. These classes are summarized with Table 4. The functional classifications were obtained from the Munich Information Center for Protein Sequences Yeast Genome Database (MYGD, http://www.mips.biochem.mpg.de/proj/yeast). MYGD class definitions come from biochemical and genetic studies of gene functions.

At this KSDG-SOM training experiment we used six functional classes from the MIPS Yeast Genome Database: tricarboxylic acid (TCA) cycle, respiration, cytoplasmic ribosomes, proteasome, histones and helix-turn-helix proteins. The first five classes represent categories of genes that on biological grounds is expected to induce similar expression characteristics. The sixth class, i.e. the helix-turn-helix proteins is used as a control group. Since there is not any biological justification for

Class	**Entropy**
1. Tricarboxylic-acid pathway (TCA)	1.96
2. Respiration-chain complexes (Resp)	1.82
3. Cytoplasmic ribosomal proteins (Ribo)	1.21
4. Proteasome (Proteas)	0.51
5. Histones	0.6
6. Helix-turn-helix	2.78
7. Rest and functionally unknown classes (Unassigned)	4.13

Table 5. The value of the entropy of class representations.

r_{su}	TCA	Resp	Ribo	Proteas	Histones
0	0.81	0.7	0.82	0.69	0.6
2	0.7	0.65	0.70	0.74	0.67
4	0.69	0.66	0.69	0.71	0.62
6	0.67	0.5	0.5	0.62	0.52
8	0.5	0.4	0.45	0.48	0.42
10	0.4	0.31	0.27	0.28	0.28
12	0.3	0.29	0.29	0.22	0.29

Table 6. Results from the reclassification experiments from the Data Set 2 with the KSDG-SOM model. The *class recovery performances* presented in this table indicate the percentage of successful "correction" of the induced classification errors by the unsupervised drive.

a mechanism that enforces the genes of this class to the same patterns of expression, we expect these genes to be spread to diverse clusters by the KSDG-SOM.

The measure *Entropy of Class Representation* is evaluated over the KSDG-SOM nodes in order to quantify the dispersion of class representation. We expect this measure to be large in the case of HTH, expressing the diversity of the HTH gene expression patterns. Indeed, the results of Table 5 support this intuitive expectation. The high entropy of the Unassigned class is due to the fact that this class accumulates hundreds of other known functional classes and all the unknown ones.

We also performed the same type of reclassification experiments as for the Data Set 1. The results of Table 6, also indicate the potentiality to recover the original classes, although at this data set the KSDG-SOM model was less effective.

Clearly, the KSDG-SOM is a tool that allows a wide-range of experiments to be performed. Since, it is dynamically growing it offers obvious benefits in comparison with static SOM approaches. In addition, the new tools that we have developed for incorporation of the a priori knowledge provide an additional potentiality for gaining new insights to biological knowledge with the sight of the microarray experiments.

9. Conclusions

This work has presented a new self-growing adaptive neural network model fitted to the requirements for clustering and classification of multi-labeled gene expression data. This model, called KSDG-SOM overcomes elegantly the main drawbacks of most of the existing clustering methods that impose an a priori specification at the number of clusters. The KSDG-SOM determines adaptively the number of clusters with a dynamic extension process which is able to exploit class information whenever available. It grows within a rectangular grid that provides the potential for the implementation of efficient training algorithms. The expansion of the KSDG-SOM is based on an adaptive process. This process grows nodes at the boundary nodes, ripples weights from the internal nodes towards the outer nodes of the grid, and inserts whole columns within the map. The growing algorithm is simple and computationally effective. It prefers to grow from the boundary nodes in order to minimize the map readjustment operations. However, a mechanism for whole column (row) insertion is implemented in order to deal with the case that a large map should be expanded around a point that is deep within its interior. The growing process determines automatically the appropriate level of expansion in order the similarity between the gene expression patterns of the same cluster to fulfill a designer definable statistical confidence level of not being a random event.

Multiple KSDG-SOM models are constructed dynamically each for a different unsupervised / supervised balance. Model selection criteria are used to select an KSDG-SOM model that optimizes the contribution of the unsupervised part of the patterns with the a priori knowledge (supervised part). The model selected on the basis of these criteria can revise better unreliable and incomplete functional labeling on the basis of unsupervised drives.

Finally, we have presented applications of the KSDG-SOM in the functional genomics area. The methodology of exploring a priori knowledge about functional gene class labeling with gene expression data, along with the adaptive expansion process, has been used to obtain clusters of genes that balance supervised with unsupervised criteria.

Future work for improvements on this technique can proceed along many possible directions. Very promising ones seem to be the incorporation of more elaborate distance metrics for the evaluation of gene expression similarity as the Bayesian distance metrics proposed in [38] and also the robustification of the KSDG-SOM clustering with techniques similar to those proposed in [7]. In addition techniques for clustering and classification of disparate types of data as those presented in [39], are being exploited in order to integrate efficiently diverse types of biological information (e.g. the known trancription factors). Much work also remains on the biological interpretation of the results and on the integration of the Java software components that implement the algorithms in a user friendly and conistent integrated environment for experimentation with gene expression data.

10. Appendix A:

The detailed algorithm of the expansion process are as follows:

<Expansion Phase:>
Computation of the GP_i, i.e. of the Growth Parameter for every node i.
    repeat
let i = the node with the maximum GP_i measure
if IsBoundaryNode(i) **then**
    // expand at the neighbours of the boundary nodes
    JoinSmoothlyNeighbours (i)
elseif IsNearBoundaryNode(i)
    RippleWeightsToNeighbours(i)
else InsertWholeColumn(i);
endif
Re-execute the Training Run Adaptation Phase for the expanded map
Reset the Growth Parameter measures for all nodes
(since possible redistribution of patterns can occur over nodes).
    until not RandomLikeClustersRemain();

We describe below shortly the main issues involved in these steps. We first explore the functionality involved in the repeat loop that controls the KSDG-SOM expansion and we concentrate on the criteria for the establishment of the proper level of expansion in the section that follows. The functions that are involved with the KSDG-SOM expansion are as follows:

- **IsBoundaryNode()** The function IsBoundaryNode() checks whether a node is a boundary node. Training efficiency and implementation simplicity were the motivations for the decision to expand mostly from the boundary nodes.
- **JoinSmoothlyNeighbours()** The expansion of the map at the boundary nodes is performed by acquiring one to three new nodes from their direct neighborhood. The weights of the new nodes are adjusted heuristically to retain the "weight flow" with the function JoinSmoothlyNeighbours().
- **IsNearBoundaryNode()**: A node is declared as a *near boundary* node by the function *IsNearBoundaryNode()* when the boundary of the map can be reached from this node by traversing in any direction at most two nodes.
- **RippleWeightsToNeighbours()**: The map configuration is slightly disturbed when the winner node is not a boundary node but is a near boundary. For a near boundary node a percentage (usually 20-50%) of the weight of the winner node is shifted towards the outer nodes with the function *RippleWeightsToNeighbours()*. This operation alters locally the Voronoi regions of influence of each Gaussian and usually with a few weight "rippling" operations the winner node is propagated to a boundary node (which is located near).

- **InsertWholeColumn()**: Finally, if the winner is a node that is neither a boundary nor a near boundary the alternative of inserting a whole empty column is used. The rippling of weights is avoided in these cases, because usually excessive computation times are required before the winner propagates from a node placed deep in the map to a boundary node. Instead of inserting whole new columns we can insert alternatively whole new rows, or we can perform a combination of row and column insertion.
- **RandomLikeClustersRemain()**: This function evaluates the "randomness" of the distribution of gene expression patterns to a specific cluster as we shall describe in the next section. If for a cluster the allocation of gene expression patterns is found to be "random", then the cluster is considered to own unrelated patterns and therefore further decomposition is required.

References

1. V. Olman, D. Xu., Y. Hu, "CUBIC: Identification of Regulatory Binding Sites Through Data Clustering", Journal of Bioinformatics and Computational Biology", p. 21-40
2. H. Liu, L. Wong, "Data Mining Tools for Biological Sequences", Journal of Bioinformatics and Computational Biology, Vol. 1, No. 1, p. 139-168, April 2003
3. Brazma Alvis, Vilo Jaak, "Gene expression data analysis", FEBS Letters, 480 (2000) 17-24
4. Eisen Michael B., Spellman Paul T., Patrick O. Brown, and David Botstein, "Cluster analysis and display of genome-wide expression patterns", Proc. Natl. Acad. Sci. USA, Vol. 95, pp. 14863-14868, December 1998
5. Ying Xu, Victor Olman, Dong Xu, "Clustering gene expression data using a graph-theoretic approach: an application of minimum spanning trees", Vol. 18, no. 4, 2002, p. 536-545
6. Mavroudi Seferina, Papadimitriou Stergios, Bezerianos Anastasios, "Gene Expression Analysis with a Dynamically Extended Self-Organized Map that Exploits Class Information", Bioinformatics, Vol. 18, no 11, 2002, p 1446-1453
7. David R. Bibkel, "Robust cluster analysis of microarray gene expression data with the number of clusters determined biologically", Bioinformatics, Vol. 19, no. 7, p. 818-824, 2003
8. Papadimitriou S., Mavroudi S., Vladutu L., Bezerianos A., "Ischemia Detection with a Self Organizing Map Supplemented by Supervised Learning", IEEE Trans. On Neural Networks, Vol. 12, No. 3, May 2001, p. 503-515
9. Herrero Javier, Valencia Alfonso, and Dopazo Joaquin, "A hierarchical unsupervised growing neural network for clustering gene expression patterns", Bioinformatics, (2001) Vol. 17, no. 2, pp. 126-136
10. Tamayo, P., Slonim, D., Mesirov, J., Zhu, Q., Kitareewan, S., Dmitrovsky, E., Lander, E.S. and Golub, T.R. (1999) "Interpreting patterns of gene expression with self-organizing maps: methods and application to hematopoietic differentiation", Proc. Natl. Acad. Sci., USA, 92, pp. 2907-2912
11. Brown Michael P.S., Grundy William Noble, Lin David, Cristianini Nello, Sugnet Charles Walsh, Furey Terrence S., Ares Manuel, Haussler Jr., David, "Knowledge-based Analysis of Microarray Gene Expression Data By Using Support Vector Machines", Proceedings of the National Academy of Science, Vol 97, No 1, pp.

262-267, 1997
12. Friedman, N., M. Linial, I. Nachman, and D'Peier, "Using Bayesian networks to analyze expression data", J. Comp. Bio. 7, 2000, 601-620,
13. Fritzke Bernd, "Growing Grid - a self organizing network with constant neighborhood range and adaptation strength", Neural Processing Letters, Vol. 2, No. 5, pp. 9-13, 1995
14. Hastie Trevor, Tibshirani Robert, Botstein David, Brown Patrick, "Supervised Harvesting of expression trees", Genome Biology 2001, 2 (1), http://genomebiology.com/2001/2/I
15. Van Hulle, N.M., "Kernel-Based Topographic Map Formation", Neural Computation, Vol. 14, No 7, p. 1560-1573, 2002
16. Van Hulle, N.M., "Kernel-based equiprobabilistic topographic map formation, Neural Computation, Vol. 10, No. 7, p. 1847-1871, 2002
17. Vesanto Juha Alhoniemi, Esa, "Clustering of the Self-Organized Map", IEEE Transactions on Neural Networks, Vol. 11, No. 3, May 2000, p. 586-600
18. Si J., Lin S., Vuong M. A., "Dynamic topology representing networks", Neural Networks, 13, pp. 617-627, 2000
19. Cheng Guojian and Zell Andreas, "Externally Growing Cell Structures for Data Evaluation of Chemical Gas Sensors", Neural Computing & Applications, 10, pp. 89-97, Springer-Verlag, 2001
20. Campos Marcos M., Carpenter Gail A., "S-TREE: self-organizing trees for data clustering and online vector quantization", Neural Networks 14 (2001), pp. 505-525
21. Van Hulle, N.M., "Joint Entropy Maximization in Kernel-Based Topographic Maps", Neural Computation, Vol. 14, No 8, p. 1887-1906, 2002
22. Azuaje Franscisco, "A Computational Neural Approach to Support the Discovery of Gene Function and Classes of Cancer", IEEE Trans. Biomed. Eng., Vol 48, No. 3, March 2001, pp 332-339
23. James R. Williamson, "Self-Organization of Topographic Mixture Networks Using Attentional Feedback", Neural Computation 13:563-593, 2001
24. Bernhard Scholkopf, Alexander J. Smola, "Learning with Kernels: Support Vector Machines, Regularization and Beyond", MIT Press 2002
25. B. Scholkopf, A. J. Smola, R. C. Williamson, P. L. Bartlett, "New support vector algorithms", Neural Computation:1207-1245, 2000
26. Janne Sinkkonen, Samuel Kaski, "Clustering Based on Conditional Distributions in an Auxiliary Space", Neural Computation, 14:217-239, 2001
27. A. Sierra, F. Corbacho, "Reclassification as Supervised Clustering", Neural Computation 12:2537-2546, 2000
28. Cheung Vivian G., Morley Michael, Aguilar Francisco, Massimi Aldo, Kucherlapati Raju, Childs Geoffrey, "Making and reading microarrays", Nature genetics supplement, Vol. 21, January 1999
29. Bishop, C. M., Svensen, M., Williams, C. K., "GTM: The generative topographic mapping", Neural Computation, 10:215-234, 1998
30. Andras, P., "Kernel-Kohonen networks", International Journal of Neural Systems, Vol. 12, No. 2 (2002) 117-135
31. Sable, Carl L. and Vasileios Hatzivassiloglou, "Text-Based Approaches for the Categorization of Images, Proceedings of the Third Annual Conference on Research and Advanced Technology for Digital Libraries, Paris, 1999. pp. 19-38.
32. Ian H. Witten, Eibe Frank, *Data Mining*, Morgan Kaufmann Publishers, 2000
33. Alahakoon Damminda, Halgamuge Saman K., Srinivasan Bala, "Dynamic Self-

Organizing Maps with Controlled Growth for Knowledge Discovery", IEEE Transactions On Neural Networks, Vol. 11, No. 3, pp 601-614, May 2000.

34. Kohonen T., Self-Organized Maps, Springer-Verlag, Second Edition,1997.
35. Haykin S,, Neural Networks, Prentice Hall International, Second Edition, 1999.
36. DeRisi, J. L., Iyer, V. R, and Brown, P.O, "Exploring the metabolic and genetic control of gene expression on a genomic scale", Science, 278, p. 680-686, 1997
37. Troyanskaya Olga, Cantor Michael, Shelock Gavin, Brown Pat, Hastie Trevor, Tibshirani Robert, Botstein David, Altman Russ B., "Missing value estimation methods for DNA microarrays", Bioinformatics, Vol. 17, no 6, 2001
38. Lawrence Hunter, Ronald C. Taylor, Sonia M. Leach and Richard Simon, "GEST: a gene expression search tool based on a novel Bayesian similarity metric", Bioinformatics, Vol. 17, Suppl. 1, p. S115-S122
39. Vasilios Petridis, Vasilis Kaburlasos, "Fuzzy Lattice Neural Network (FLNN): A Hybbrid Model for Learning", IEEE Transactions on Neural Networks, Vol. 9, No 5, Semptember 1998

Fig. 1. The results of the data shuffling illustrate that the distances between the randomized data occupy a distinct distribution. For the example case of the gene expression data positive correlation is favored while for the random case the distribution has a normal form. The analysed data concern the 6,221 genes from the budding yeast Saccharromyces cerevisiae public available from the Stanford web site and functionally labeled as described in Section 8.

Fig. 2. The selection of an appropriate parameter for model selection that accounts the a priori knowledge. The analysed data concern the 6,221 genes from the budding yeast Saccharromyces cerevisiae public available from the Stanford web site and functionally labeled as described in Section 8, i.e. the same dataset used also in Figure 1.

Fig. 3. The average consistency ratio for some clustering algorithms compared with KSDG-SOM.

Fig. 4. A snapshot of KSDG-SOM training